LAB 3

File Inclusion & SQL Injection Security Assessment Report

DVWA and OWASP Mutillidae II — Isolated Kali Linux VM (VMware NAT)

Student Name	Daniyal Ahmed
Registration Number	C11/26/EHIT/17331
Cohort	Ethical Hacking Internship Training
Course	Offensive Security Operations I (CIP-A104)
Submission Date	31 August 2026
Assessment	Lab 3

Authorised training assessment performed exclusively against locally installed, intentionally vulnerable applications inside an isolated VM. No public exposure. All evidence limited to benign markers and instructor-designated fictional data.

Contents

1. Executive Summary

2. Scope and Rules of Engagement

3. Methodology Overview

4. Phase 1 — File-Selection Baseline

5. Phase 2 — LFI Validation

6. Phase 3 — RFI Validation (Scoped Out)

7. Phase 4 — File-Inclusion Secure Comparison

8. Phase 5 — SQL Baseline

9. Phase 6 — SQLi Discovery (Boolean/Behavioural)

10. Phase 7 — Authentication / Marker-Data Proof

11. Phase 8 — Security Comparison (SQLi)

12. Analysis Questions

13. Remediation Recommendations

14. Evidence Register

15. Evidence-Minimisation Statement

16. Cleanup and Restoration Log

17. Conclusion

1. Executive Summary

This assessment evaluated two categories of server-side input weaknesses across DVWA and OWASP Mutillidae II: unsafe resource selection (Local File Inclusion) and unsafe database queries (SQL Injection). Both applications were found vulnerable to LFI and SQLi at their lowest security settings, with distinct architectural differences in how each application handles the same class of vulnerability.

A key finding of this assessment is that the two applications' “secure modes” are not equivalent in practice. DVWA's High security level for both LFI and SQLi applies surface-level controls (filename pattern matching; session-based parameter delivery with a LIMIT clause) that do not address the underlying root cause — unsanitized string concatenation into filesystem and SQL operations. Mutillidae's Secure level (5), by contrast, appears to implement genuine input-handling controls that fully blocked the same attack classes without disclosing errors.

Vulnerability	DVWA (Low)	DVWA (High)	Mutillidae (Level 0)	Mutillidae (Level 5)
LFI	Exploitable	Blocked (whitelist)	Exploitable	Blocked (whitelist)
SQLi (boolean)	Exploitable	Still exploitable	Exploitable	Blocked
SQLi (auth bypass)	Exploitable	Exploitable (row-capped)	Exploitable	Blocked
Verbose errors	Suppressed	Suppressed	Full SQL error + stack trace	Suppressed

2. Scope and Rules of Engagement

Testing was performed exclusively against locally installed instances of DVWA (127.0.0.1:42001) and OWASP Mutillidae II (127.0.0.1:42002), running inside an isolated Kali Linux VM on VMware NAT. Neither application was exposed publicly at any point.

- File inclusion: DVWA File Inclusion module and Mutillidae LFI/RFI challenges.
- SQL Injection: DVWA SQL Injection / SQL Injection (Blind) and Mutillidae SQLi User Lookup challenge.
- Evidence limited to benign local markers and one instructor-designated fictional database marker (Mutillidae's built-in “admin” demo account).
- Prohibited and not performed: OS secret access, private key access, remote code execution, use of external servers, database dumping, credential collection beyond app-disclosed demo data, and unrelated table enumeration.

3. Methodology Overview

Traffic was intercepted and analysed using Burp Suite Community Edition (Proxy, Intercept, Repeater, and HTTP History). Testing followed the nine required hands-on phases in sequence: baseline capture, LFI validation, RFI

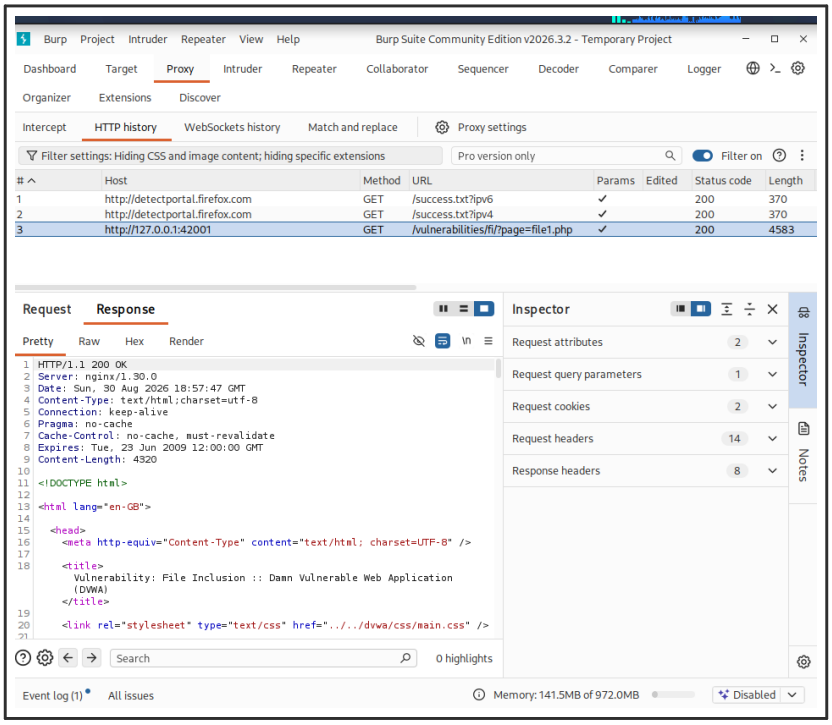
validation (conditional), secure-mode comparison, SQL baseline, boolean/behavioural SQLi discovery, minimal authentication/marker-data proof, a second secure-mode comparison for SQLi, and remediation planning. At each stage, evidence was captured as matched request/response pairs, and behavioural signatures (response length, message text) were used as the primary observables for injection proof, in line with minimal-evidence requirements.

Phase 1 — File-Selection Baseline

Goal: capture normal, valid file-selection requests from both applications and identify the controlling parameter.

DVWA (127.0.0.1:42001)

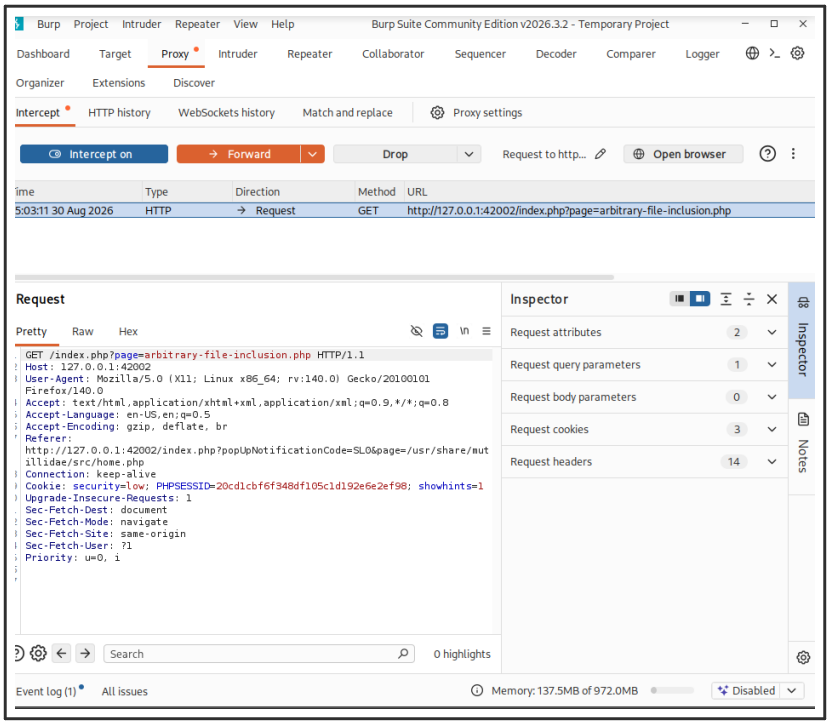
The controlling parameter is **page** on **/vulnerabilities/fi/**, accepting a bare relative filename (e.g. `file1.php`) from an implicit whitelist context. A normal request (`?page=file1.php`) returned HTTP 200 with Content-Length 4583.



Evidence 1.1 — DVWA baseline request/response for page=file1.php (200 OK, Content-Length 4583).

Mutillidae (127.0.0.1:42002)

The controlling parameter is also **page**, on **/index.php**. Critically, the **Referer header** on this baseline request revealed that Mutillidae's own internal navigation had just used `page=/usr/share/mutillidae/src/home.php` — a full absolute filesystem path — as part of ordinary use. This is a significant trust-boundary observation: Mutillidae treats **page** as a raw include target even in legitimate baseline traffic, whereas DVWA's baseline conceals the underlying path mechanism behind short filenames.



Evidence 1.2 — Mutillidae baseline request intercepted in Burp; Referer discloses an absolute filesystem path.

Phase 2 — LFI Validation

Goal: prove unintended path control using a harmless local marker file, and record path-resolution / error-disclosure behaviour.

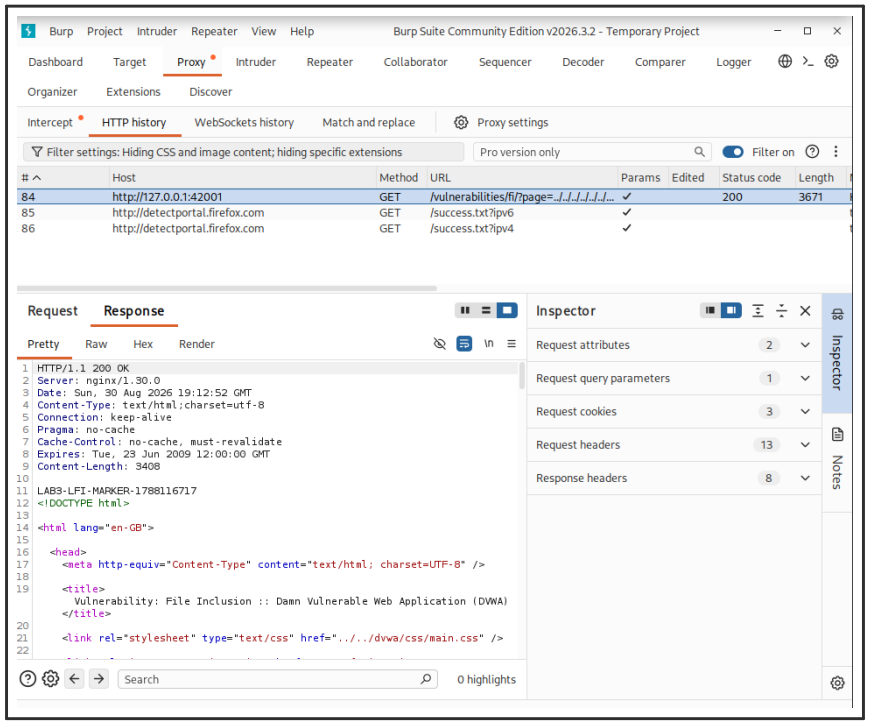
A benign marker file was created directly on the VM's disk (not through either web application): `/tmp/lfi_marker.txt`, content `LAB3-LFI-MARKER-1788116717`, SHA256 `9af243da...43952a`.



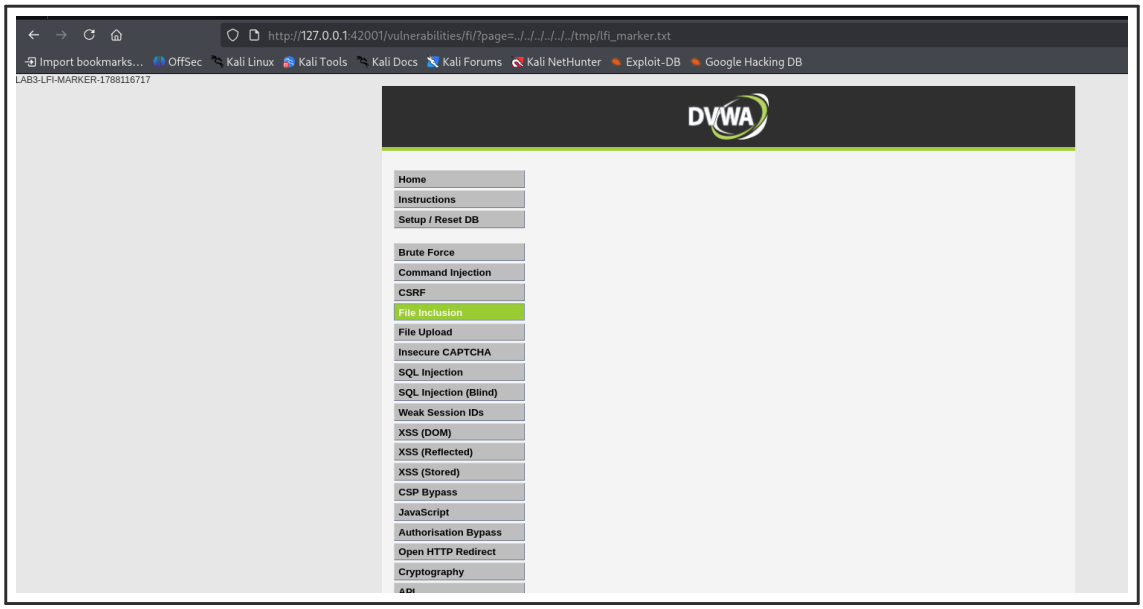
Evidence 2.1 — Marker file creation and SHA256 hash, captured off-application on the VM.

DVWA — LFI Success

Request: `GET /vulnerabilities/fi/?page=../../../../../../tmp/lfi_marker.txt` (6 levels of traversal). Response: HTTP 200, Content-Length 3671. The marker string was returned verbatim, rendered *above* DVWA's own page template — proof of server-side `include()` execution, not a browser redirect, since the page chrome and URL bar never changed.



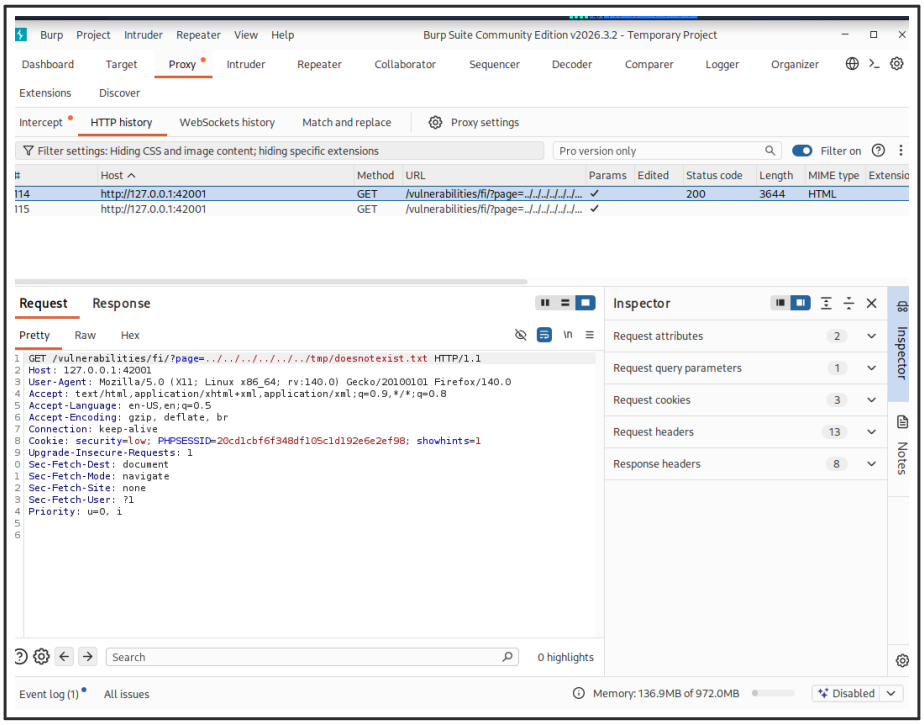
Evidence 2.2 — DVWA traversal request/response; marker content returned inside the response body.



Evidence 2.3 — Browser render confirming marker text above the DVWA page header.

DVWA — Error / Disclosure Case

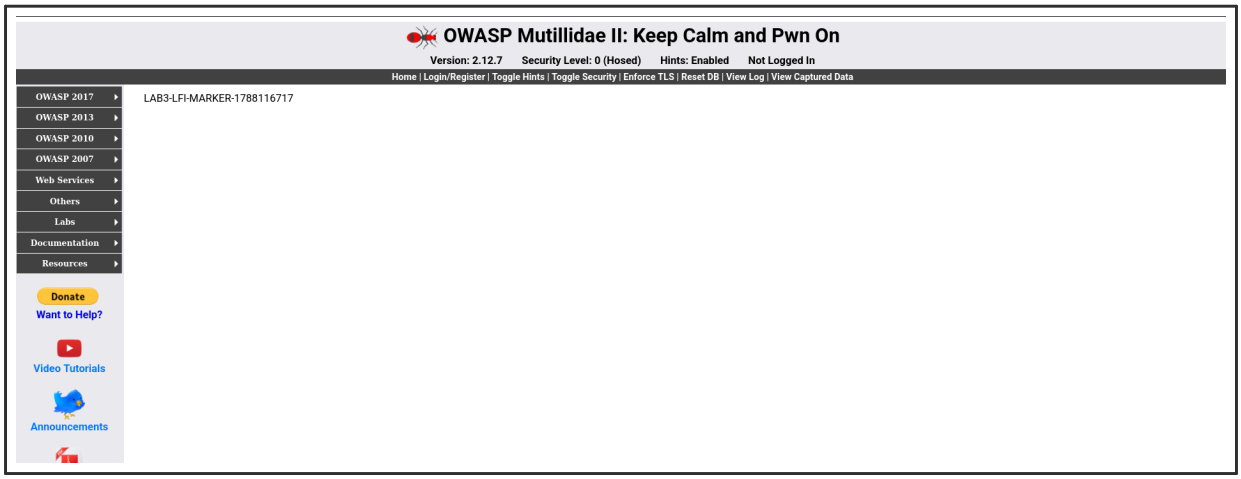
Requesting a nonexistent path (`../../../../../../tmp/doesnotexist.txt`) returned HTTP 200 with an **empty** main-body placeholder — no PHP warning, no stack trace, no resolved path leaked. This confirms `display_errors` is suppressed even at low security, while the traversal mechanism itself still functions for files that do exist.



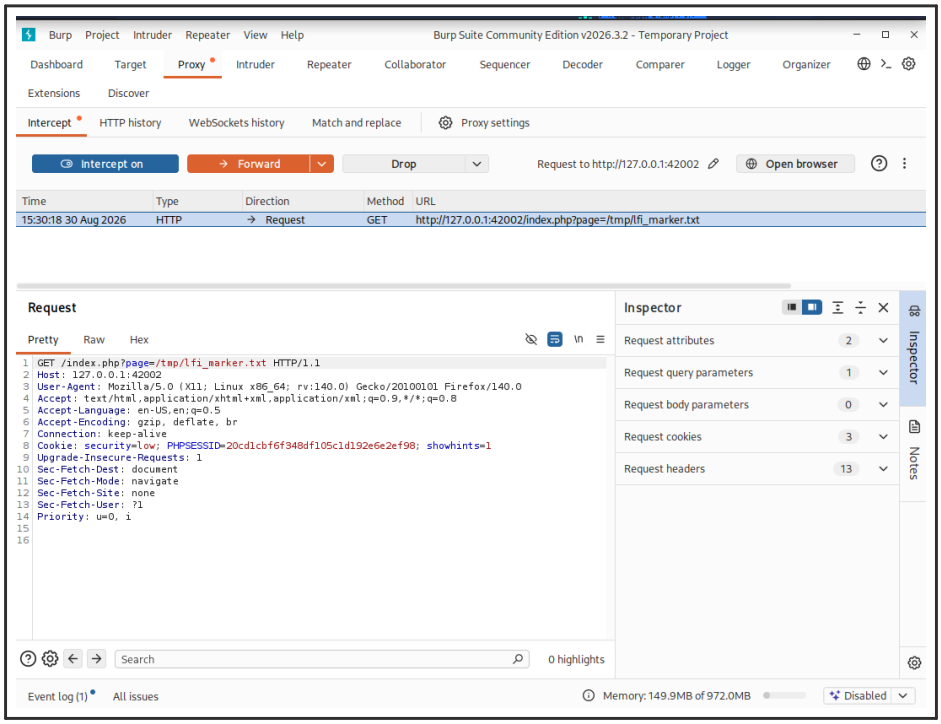
Evidence 2.4 — DVWA silent failure for a nonexistent traversal target (blank body, no disclosure).

Mutillidae — LFI Success

Consistent with the Phase 1 observation, no traversal was required at all: `GET /index.php?page=/tmp/lfi_marker.txt` returned HTTP 200 with the marker string rendered directly in the page content.



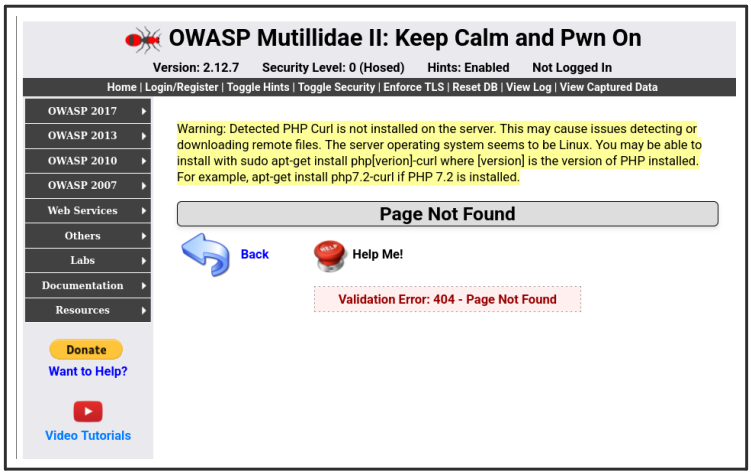
Evidence 2.5 — Mutillidae marker content rendered directly via absolute-path inclusion.



Evidence 2.6 — Burp Intercept capture of the Mutillidae absolute-path LFI request.

Mutillidae — Error / Disclosure Case

A nonexistent absolute path returned a custom application-level “Page Not Found / Validation Error: 404” panel rather than a raw PHP include() warning — a different failure mode than DVWA's silent blank output, but similarly free of raw disclosure at this stage.



Evidence 2.7 — Mutillidae custom 404/validation error panel for a nonexistent include target.

Phase 3 — RFI Validation — Scoped Out

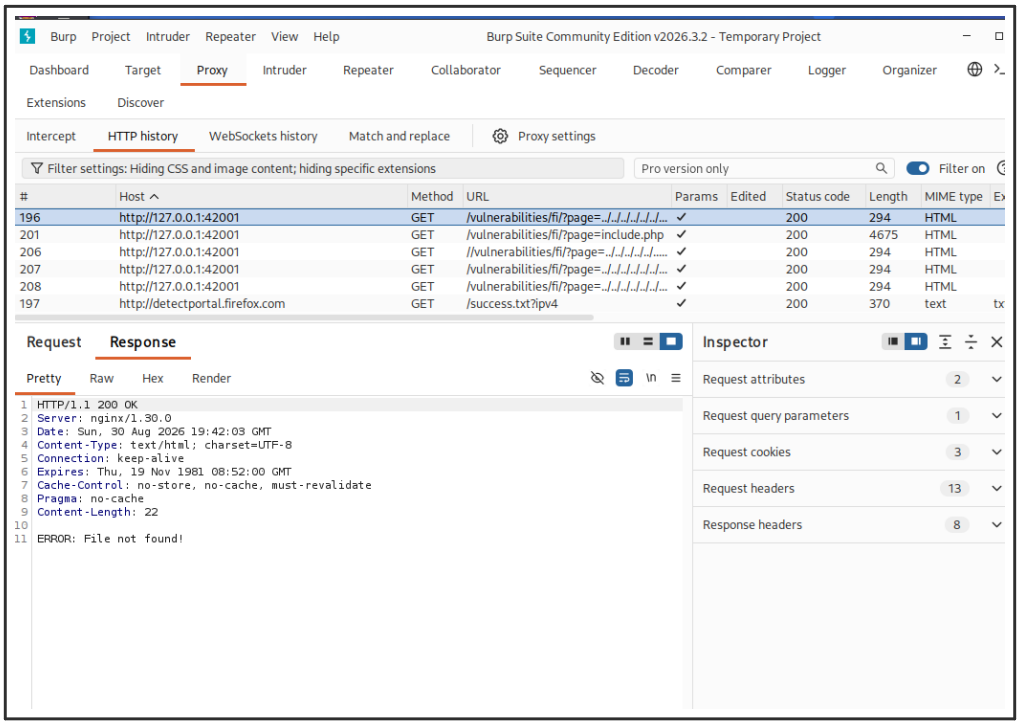
RFI validation is required only where deliberately enabled. `allow_url_include` was not confirmed enabled in this environment, and Mutillidae additionally surfaced an environment banner indicating PHP Curl was not installed — a dependency for several of its URL-fetch code paths. Per lab scope, RFI was not attempted; no local static marker service was stood up, and this phase is documented as skipped rather than forced against an unconfigured target.

Phase 4 — File-Inclusion Secure Comparison

Goal: repeat the LFI test at a stronger security level and document the specific control that prevents inclusion.

DVWA (High Security) — Blocked

The identical traversal payload now returned HTTP 200, Content-Length 294, body: `ERROR: File not found!`. The whitelisted file (`page=include.php`) continued to function normally (200, 4675 bytes), and repeated traversal attempts consistently produced the same 294-byte error. **Mechanism:** a filename pattern/whitelist match on the resolved `page` value — an allow-list on filename shape.



Evidence 4.1 — DVWA High security consistently blocks traversal with a generic error.

Mutillidae (Security Level 5 / Secure) — Blocked

The same absolute-path payload that succeeded at Level 0 now returned “Validation Error: 404 — Page Not Found” — even though the target file genuinely exists on disk. **Mechanism:** at Secure level, `page` is validated against an internal registry of legitimate page identifiers, independent of file existence — an allow-list on identity, which is structurally stronger than DVWA's shape-based check.



Evidence 4.2 — Mutillidae Security Level 5 blocking the previously-successful LFI marker request.

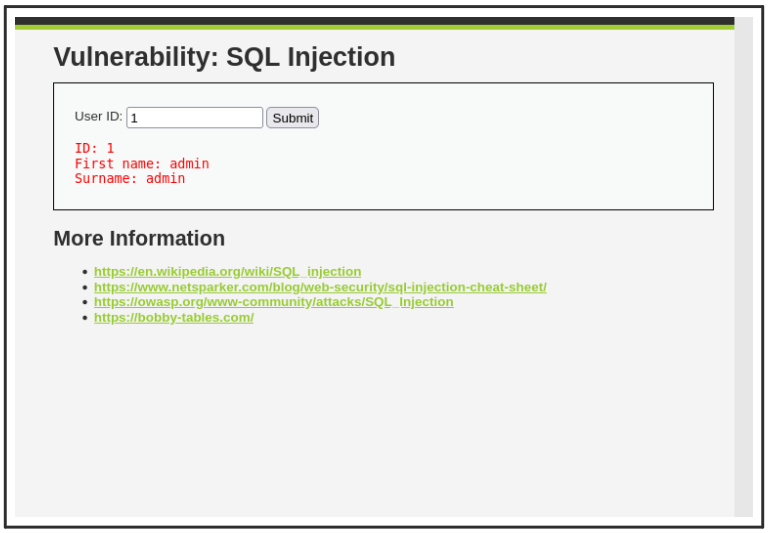
	Low / Hosed	High / Secure
DVWA	Free traversal to any readable file	Blocked via filename pattern match
Mutillidae	Free absolute-path inclusion, no traversal needed	Blocked via page-identity whitelist

Phase 5 — SQL Baseline

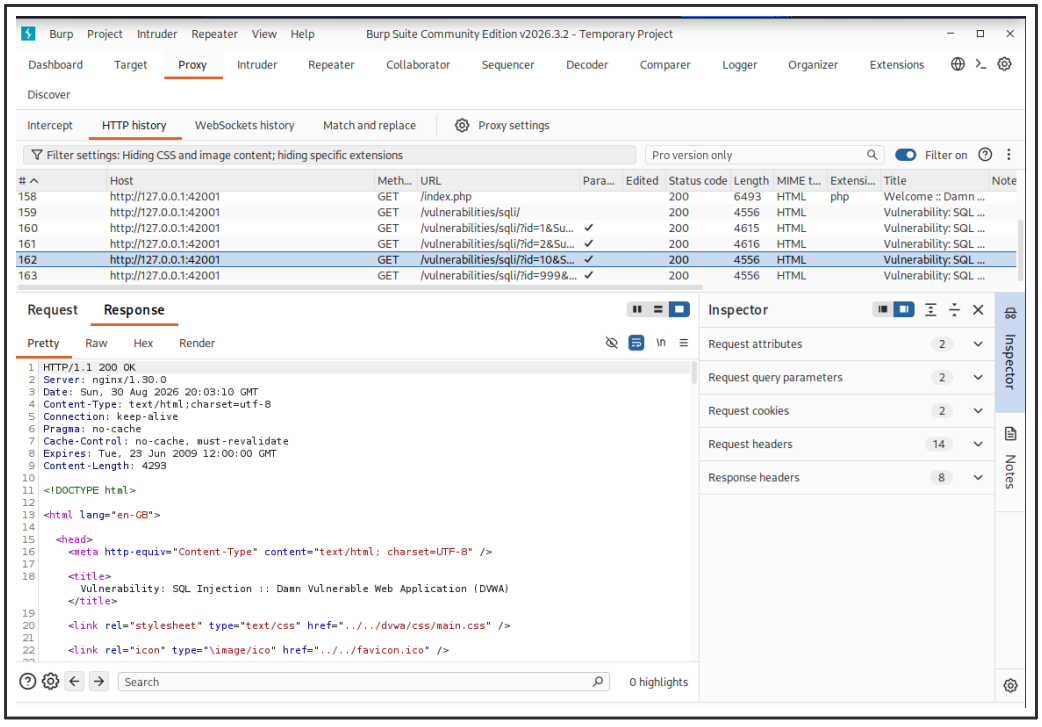
Goal: capture normal valid and invalid database-backed requests in both applications before attempting injection.

DVWA

Valid lookup (`id=1`) returned the admin record. Invalid lookups (`id=999`, `id=10`) returned an identical Content-Length of 4556 — a consistent “no-match” signature, distinct from the larger, name-dependent lengths of valid matches (4615–4627 bytes). This length delta became the primary observable for the boolean tests in Phase 6.



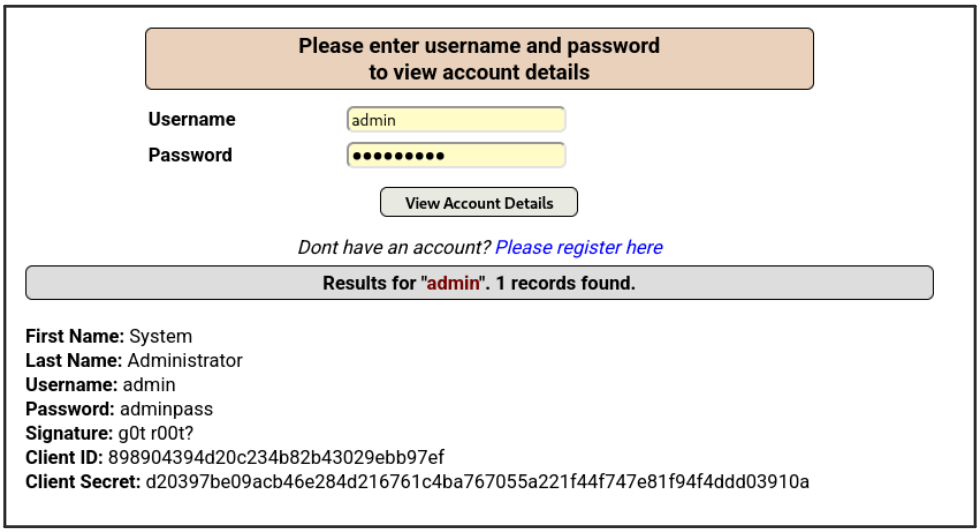
Evidence 5.1 — DVWA valid lookup (id=1) returning the admin demo record.



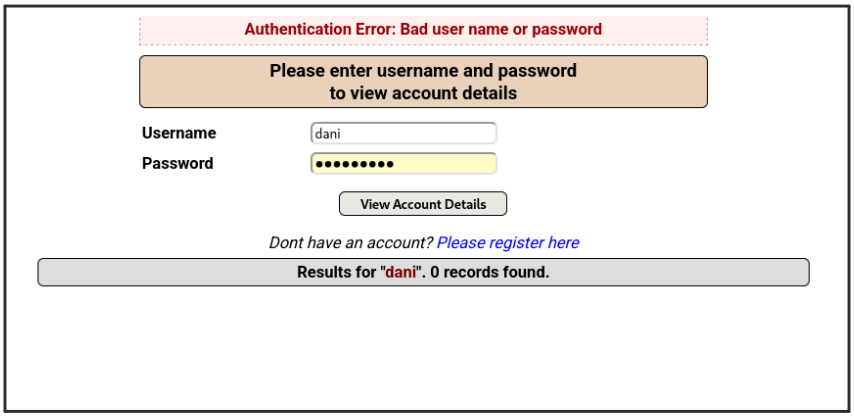
Evidence 5.2 — DVWA HTTP history showing the found/not-found length signature across several IDs.

Mutillidae

Valid lookup (`username=admin`, correct password) returned “Results for 'admin'. 1 records found.” with the full fictional demo record. An invalid lookup returned “0 records found” plus an additional “Authentication Error” banner — a much more visibly distinguishable baseline than DVWA's near-identical page structure.



Evidence 5.3 — Mutillidae valid lookup returning the built-in fictional admin demo record.



Evidence 5.4 — Mutillidae invalid lookup: “0 records found” with an authentication error banner.

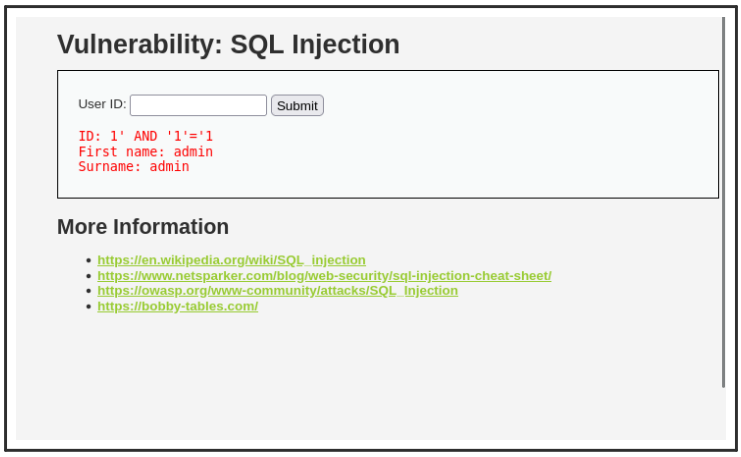
Phase 6 — SQLi Discovery (Boolean / Behavioural)

Goal: determine whether input changes the query logic itself, not merely triggers a parser error.

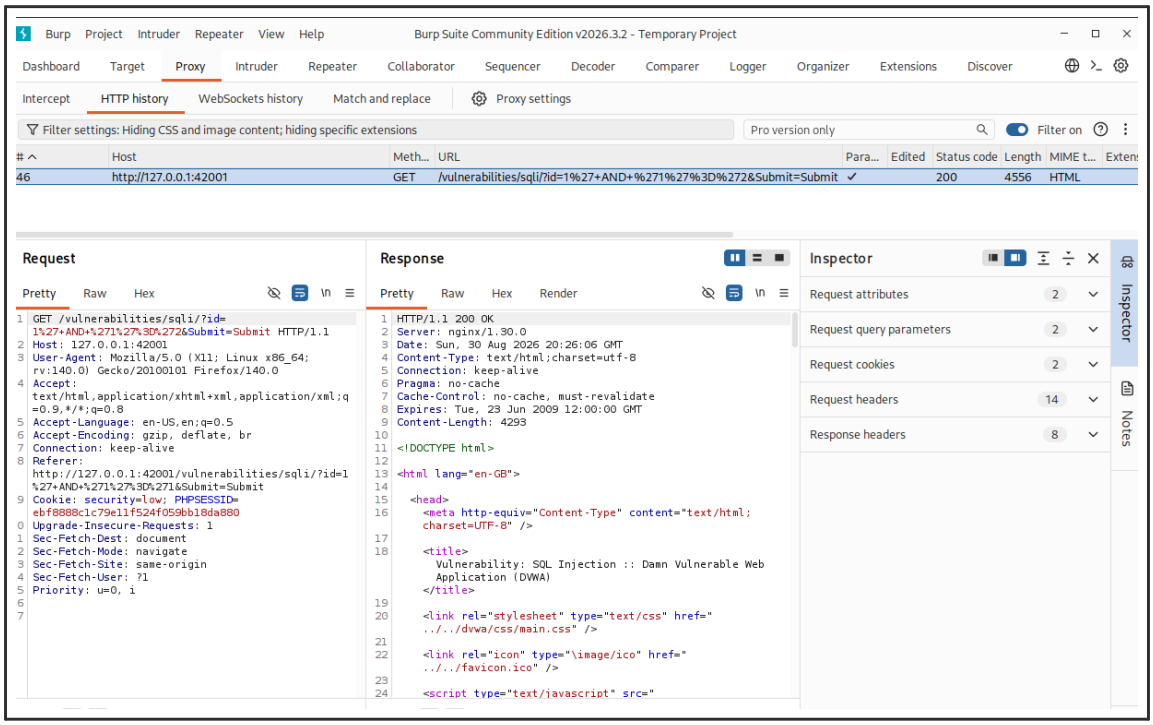
Methodology note: initial attempts mistakenly included a literal “id=” prefix inside the payload value itself, which broke the base comparison regardless of the injected clause's truth value (both variants returned the same “not found” length). This was corrected by submitting only the bare payload into the input field, isolating the injected clause as the sole variable.

DVWA

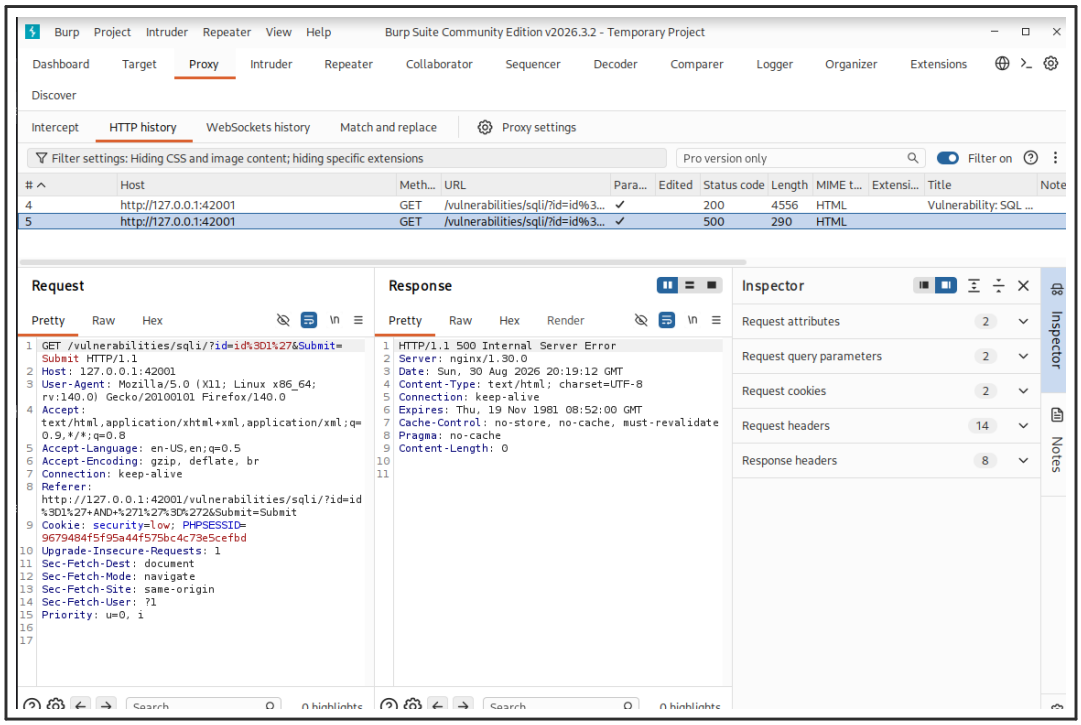
True condition (`1' AND '1'='1`): HTTP 200, Content-Length 4627 — record found. **False condition** (`1' AND '1'='2`): HTTP 200, Content-Length 4556 — matches the established “no match” baseline exactly. **Malformed case** (single quote alone): HTTP 500, blank body — a raw syntax break, distinct from the logic-controlled result above.



Evidence 6.1 — DVWA true-condition boolean test: record found.



Evidence 6.2 — DVWA false-condition boolean test: response length matches the “not found” baseline.

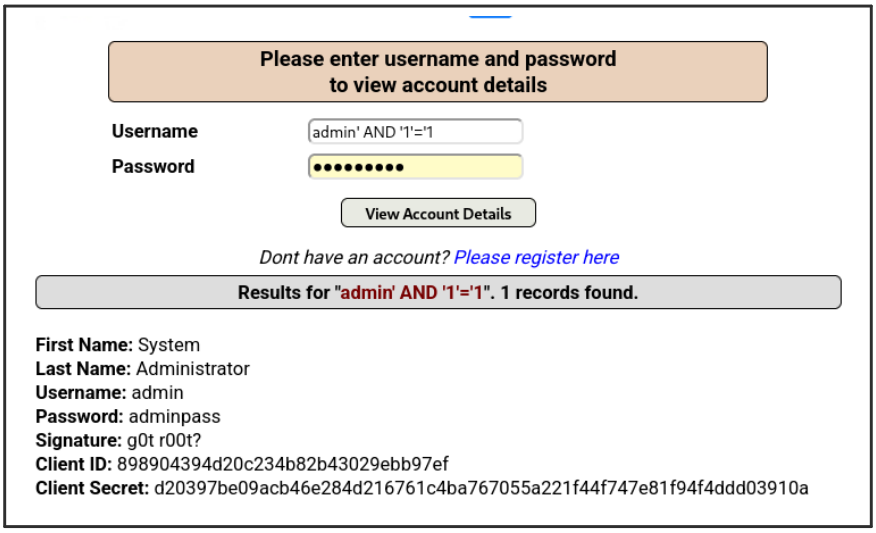


Evidence 6.3 — DVWA malformed single-quote input causing a raw HTTP 500 (syntax break, not logic control).

Conclusion: identical base ID and identical injected syntax shape, with only the AND clause's truth value varied, produced a deterministic flip between found/not-found states — proof the input is concatenated directly into the query and evaluated as logic.

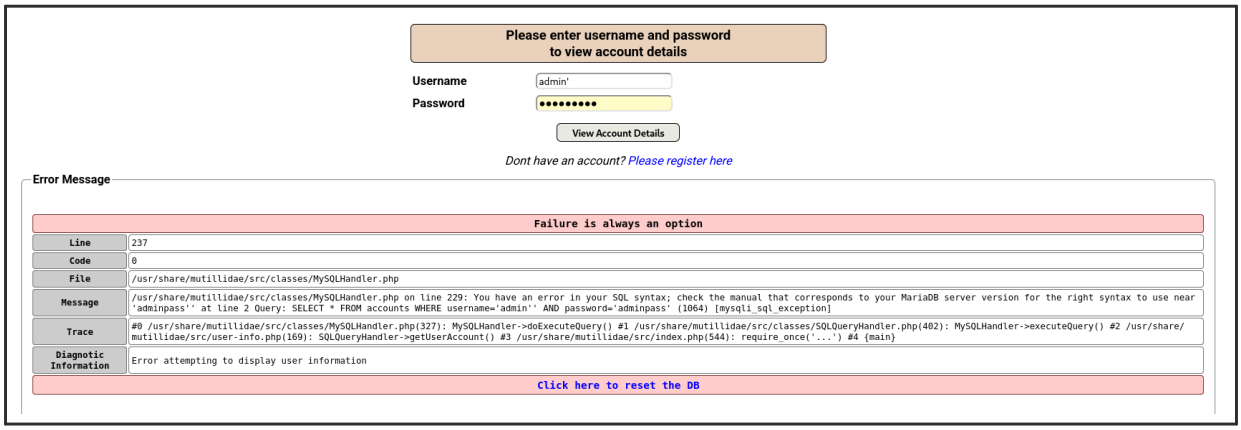
Mutillidae

True condition (`admin' AND '1'='1`, correct password): HTTP 200, “1 records found”, full record returned.
False condition (`admin' AND '1'='2`, correct password): smaller response, consistent with “0 records found”. A control check with a clean baseline (no injection) confirmed the failure was caused specifically by the injected clause, not the password.



Evidence 6.4 — Mutillidae true-condition boolean test: full admin record returned.

The malformed single-quote case produced a **full raw application error disclosure**, significantly more severe than DVWA's silent failure: the exact query structure, database engine, server file path, line number, and complete call stack were all leaked.



Evidence 6.5 — Mutillidae raw MySQL error disclosure: full query, file path, line number, and stack trace.

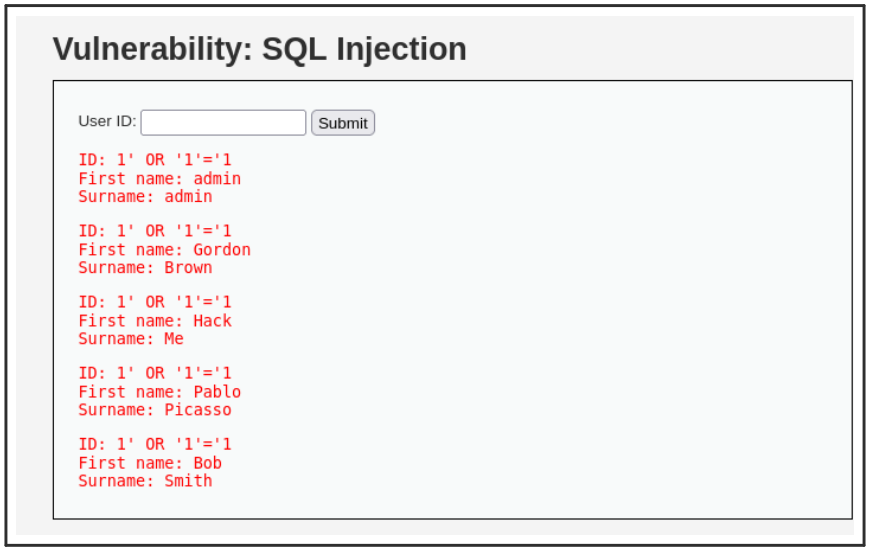
Conclusion: the same boolean-flip proof holds for Mutillidae. Additionally, this verbose error is a standalone, high-severity information-disclosure finding in its own right.

Phase 7 — Authentication / Marker-Data Proof

Per lab scope, this phase uses only a designated lab account or fictional marker record, and evidence collection stopped immediately after minimal proof was captured — no further enumeration was performed in either case.

DVWA — OR-Based Multi-Record Proof

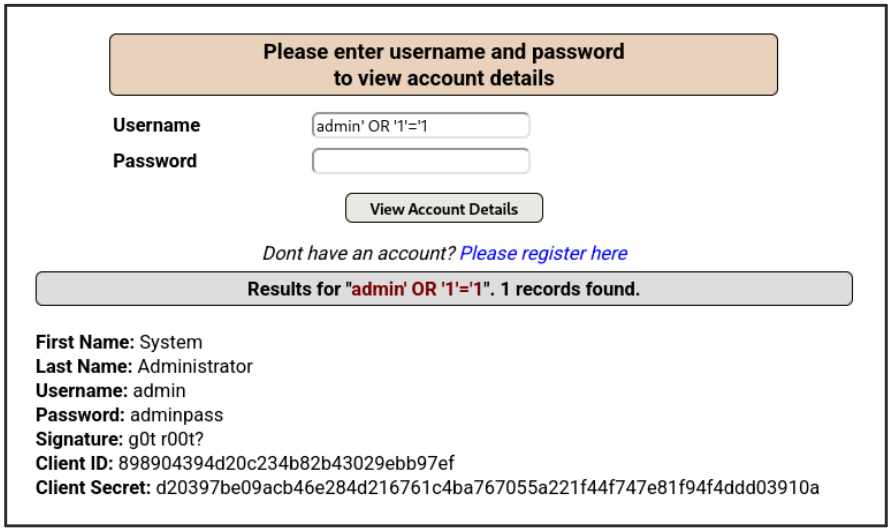
`id=1' OR '1'='1` returned **5 records** (all of DVWA's own built-in demo users) instead of the single row a legitimate `id=1` query returns — a classic OR-based logic bypass.



Evidence 7.1 — DVWA OR-based injection returning all demo users instead of one.

Mutillidae — Authentication Bypass Proof

`username=admin' OR '1'='1` with the password field left **blank** returned the full admin record — a genuine authentication bypass, since no correct password was ever supplied.

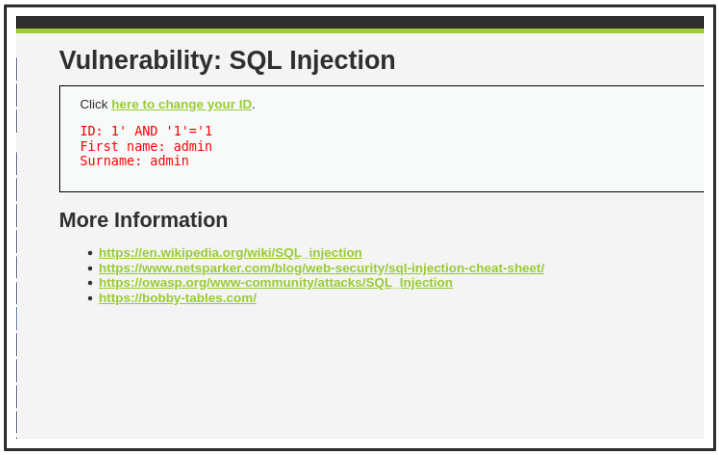


Evidence 7.2 — Mutillidae authentication bypass: admin record returned with no password supplied.

Phase 8 — Security Comparison (SQLi, Stronger Mode)

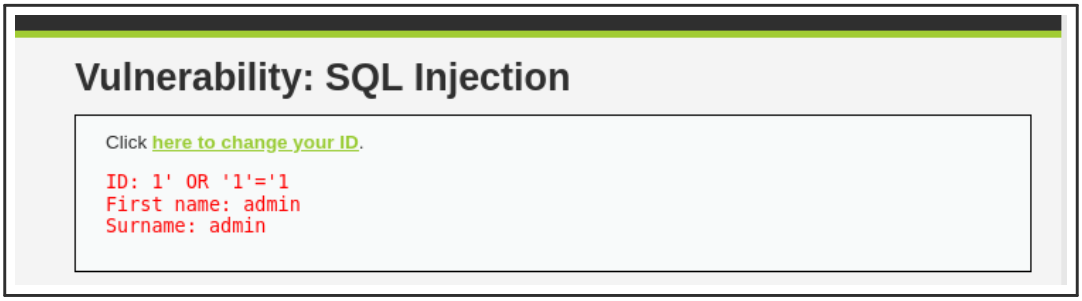
DVWA (High Security) — Mechanism Changed, Injection Still Works

High security routes the SQLi module through a two-step session-based mechanism: an ID is first POSTed to `session-input.php` and stored server-side, then a separate GET renders the result from session state — removing simple URL-parameter tampering. However, the boolean true-condition payload, submitted via this POST path, **still succeeded**: the admin record was returned exactly as at Low security.



Evidence 8.1 — DVWA High security: boolean injection still succeeds via the session-input POST path.

An OR-based bypass at High security returned only **one** record (not five, as at Low) — confirming an added `LIMIT 1` clause caps the visible result set, even though the underlying WHERE clause is still fully hijacked. This is a key finding for tool validation: an automated scan seeing a single, normal-looking result here could easily conclude the endpoint is safe, when the query logic was in fact bypassed.

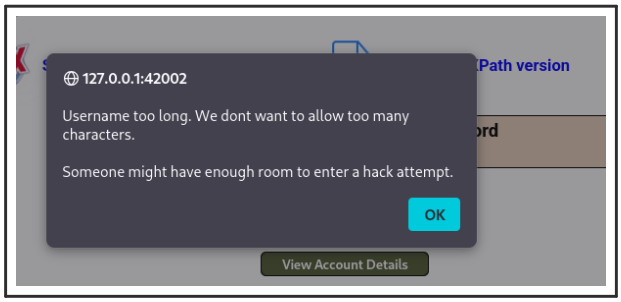


Evidence 8.2 — DVWA High security: OR bypass still succeeds but is capped to one visible row (LIMIT 1).

Finding: DVWA's High security level does not sanitise or parameterise the query — it only changes the delivery mechanism and result-set size. The root cause (string concatenation) remains unaddressed.

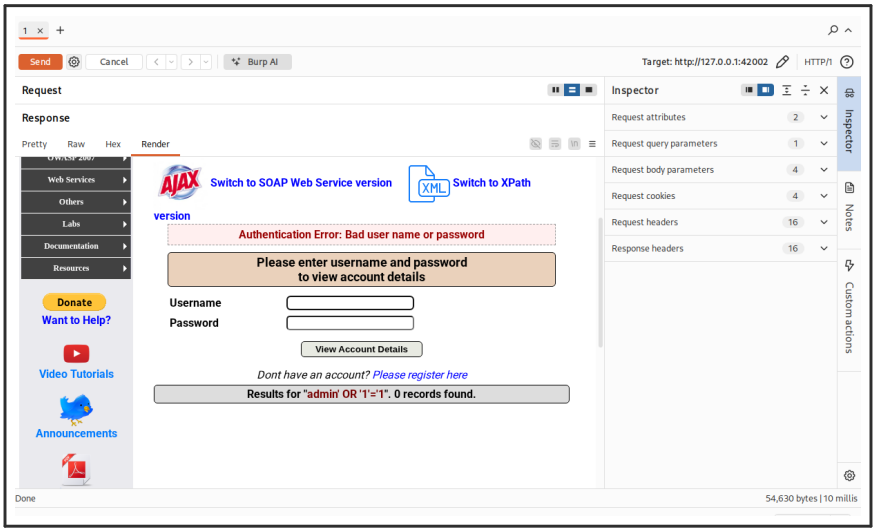
Mutillidae (Security Level 5 / Secure) — Blocked

Level 5 adds client-side JavaScript length validation on the login form, and switches the request method from GET to POST. The JS check was bypassed by submitting a short placeholder value through the browser, capturing the resulting request in Burp, and editing the raw parameter in Repeater before sending — isolating the server-side control from the front-end one.



Evidence 8.3 — Mutillidae Level 5 client-side length validation (bypassed via Burp Repeater, not a server-side control).

The same OR-based bypass payload that succeeded at Level 0 now returned “Authentication Error: Bad user name or password” and “0 records found” — with **no raw SQL error disclosed** this time either, unlike the Level 0 malformed-input case. This suggests genuine parameterised query handling rather than mere error suppression.



Evidence 8.4 — Mutillidae Level 5: OR-based bypass fully blocked, no record returned, no error disclosed.

Finding: Mutillidae's Secure level represents a genuine architectural fix for this vulnerability class.

	Weak Level	Strong Level	Verdict
DVWA	Full injection (Low)	Injection still works via session-input POST; only URL-tampering and row count mitigated (High)	Not actually fixed
Mutillidae	Full injection + raw SQL error disclosure (Level 0)	Injection blocked, no error disclosed (Level 5)	Effectively fixed

This asymmetry is a key finding: security “levels” in these training applications are not uniformly meaningful. An automated scanner run only at default settings, or one that does not manually validate found/not-found semantics and row-count artefacts, could easily mis-rate either application's true risk.

12. Analysis Questions

How is server-side file inclusion distinguished from a browser redirect?

In every successful LFI case (Evidence 2.2–2.3, 2.5–2.6), the marker content was rendered *inside* the application's own page template — appearing above or within the existing page chrome, with the browser's URL bar and navigation state unchanged. A browser redirect would instead produce a new HTTP response with a different URL, a 3xx status code, or a full page reload to a different resource. Server-side inclusion via PHP's `include()` merges file content into the current response at the point of inclusion, which is exactly what was observed: the marker text sits alongside DVWA's/Mutillidae's own header, menu, and footer elements in a single HTTP 200 response.

Why is a harmless marker sufficient to prove path control?

The marker file (Evidence 2.1) was created independently of either web application, with a unique, timestamped, hashed value that could not already exist anywhere in either app's codebase or database. Its appearance in an HTTP response therefore proves the application read and returned content from an attacker-chosen filesystem location — the same underlying mechanism an attacker would use to reach sensitive files (configuration files, credentials, source code). Because the marker is inert and disposable, this proof is achieved without ever touching real secrets, satisfying the lab's evidence-minimisation requirement while remaining fully conclusive.

What shows SQL input changed query logic rather than merely causing an error?

The boolean pair in Phase 6 (Evidence 6.1–6.2 for DVWA; Evidence 6.4 plus its false-condition counterpart for Mutillidae) holds the base value and injected syntax shape constant, varying only the truth value of an appended `AND` clause. A deterministic flip between “found” and “not found” results — with no error in either case — proves the database evaluated the injected clause as part of its logic. This is categorically different from the malformed single-quote tests (Evidence 6.3, 6.5), which merely broke query syntax and produced an error; a syntax error shows input reaches the query, but does not by itself prove an attacker can control what the query returns.

Why are fixed resource mappings and prepared statements stronger than denylists?

Both controls observed in this assessment (Phase 4's filename/page whitelists; Mutillidae's apparent Level 5 query handling) worked by allow-listing a known-good set of values or query shapes, not by attempting to blacklist dangerous characters or sequences. Denylists (blocking `../`, quotes, or SQL keywords) are inherently incomplete: they require anticipating every encoding, case variation, or syntactic trick an attacker might use, and DVWA's own High-security SQLi module (Evidence 8.1) demonstrates this failure mode — a partial control (session-based delivery, LIMIT 1) left the root vulnerability intact. Fixed resource mappings remove user input from the filesystem trust boundary entirely, and prepared statements separate SQL code from data at the protocol level, so injected syntax is treated as inert data rather than executable logic — a structural fix rather than an attempt to filter a dangerous input space.

13. Remediation Recommendations

Vulnerability	Weak Control Observed	Recommended Fix	Why It's Stronger
LFI (DVWA)	Filename pattern/whitelist on 'page'	Fixed resource mapping — map short keys to hardcoded absolute paths server-side; never pass user input into an include function	Removes the parameter from the trust boundary entirely
LFI (Mutillidae)	Page-identity whitelist (Level 5)	Same fixed mapping plus canonicalisation (realpath() + base-directory prefix check) as defense in depth	Ensures even a whitelist bug cannot escape the intended directory
SQLi (DVWA)	Session-based delivery + LIMIT 1, but raw concatenation retained	Prepared statements / parameterised queries — bind 'id' as a parameter, never concatenate	Makes injected syntax structurally inert, not just harder to reach
SQLi (Mutillidae)	Level 5 blocks injection without error disclosure	Confirm and extend parameterised queries to every query path, not just this one form	One protected endpoint does not guarantee app-wide consistency
Error handling (both)	DVWA: silent failure; Mutillidae Level 0: full raw SQL error + stack trace	Generic client-facing errors; detailed errors logged server-side only	Prevents DB engine, file structure, and query-shape fingerprinting
Least privilege	Not directly tested	DB account used by both apps should hold only minimum required grants (SELECT on relevant tables; no FILE/DROP/admin)	Bounds blast radius even if injection is later found

14. Evidence Register

#	Source	Action	Purpose
E1	DVWA	GET /vulnerabilities/fi/?page=file1.php	Phase 1 baseline
E2	Mutillidae	GET /index.php?page=arbitrary-file-inclusion.php	Phase 1 baseline
E3	VM (local)	Created /tmp/lfi_marker.txt + SHA256	Phase 2 benign marker
E4	DVWA	GET ?page=../../../../../../tmp/lfi_marker.txt	Phase 2 LFI success
E5	DVWA	GET ?page=...doesnotexist.txt	Phase 2 error/disclosure case
E6	Mutillidae	GET ?page=/tmp/lfi_marker.txt	Phase 2 LFI success (no traversal)
E7	Mutillidae	GET ?page=/tmp/doesnotexist.txt	Phase 2 error/disclosure case
E8	DVWA	GET ?page=...marker.txt (security=high)	Phase 4 blocked (whitelist)
E9	Mutillidae	GET ?page=/tmp/marker.txt (Level 5)	Phase 4 blocked (page-identity whitelist)
E10	DVWA	GET ?id=1 / ?id=999	Phase 5 SQL baseline
E11	Mutillidae	username=admin / username=dani	Phase 5 SQL baseline
E12	DVWA	id=1' AND '1'='1 / ='2	Phase 6 boolean discovery
E13	DVWA	id=1'	Phase 6 malformed/error case (HTTP 500)
E14	Mutillidae	username=admin' AND '1'='1 / ='2	Phase 6 boolean discovery
E15	Mutillidae	username=admin'	Phase 6 raw SQL error disclosure
E16	DVWA	id=1' OR '1'='1	Phase 7 OR-based multi-record proof
E17	Mutillidae	username=admin' OR '1'='1, password blank	Phase 7 authentication bypass
E18	DVWA	session-input POST, AND/OR payloads (security=high)	Phase 8 secure comparison
E19	Mutillidae	OR-bypass via Repeater (Level 5)	Phase 8 secure comparison

15. Evidence-Minimisation Statement

This assessment strictly limited evidence collection to the minimum required to demonstrate each vulnerability class, per lab scope.

- File inclusion: proof relied on a single benign local marker file created off-application directly on the VM's disk, with a recorded SHA256 hash. No OS secrets, configuration files, credential stores, or private keys were targeted at any point.
- SQL Injection: all logic-manipulation tests used only the applications' own built-in, intentionally fictional demo data. No database dumping, no credential harvesting beyond what the vulnerable applications already voluntarily disclose for these fictional accounts, and no enumeration of unrelated tables or schemas. Where an OR-based bypass could have returned an entire table (DVWA Phase 7), the test was captured once and evidence collection stopped immediately.
- RFI: not attempted. allow_url_include capability was not confirmed enabled, and a missing PHP Curl dependency further indicated the code path was not viable; Phase 3 was documented as scoped-out rather than forced.
- No content was exfiltrated outside the isolated VM/NAT environment. All testing was confined to 127.0.0.1:42001 and 127.0.0.1:42002.

16. Cleanup and Restoration Log

- Delete local marker file: rm -f /tmp/lfi_marker.txt
- Confirm no other temporary marker/export files remain in /tmp
- Reset DVWA Security level to Low (default lab baseline)
- Reset Mutillidae Security Level to 0 / Hosed (default lab baseline)
- Use Mutillidae's built-in Reset DB function to restore default demo state
- Use DVWA's Setup / Reset DB function to confirm/restore default demo state
- Confirm no database dump files were created or persisted
- Verify both applications respond normally after reset
- Archive or clear the Burp Suite project file as appropriate for submission
- Confirm no local marker/static file server was started (RFI phase skipped) — no process to stop

17. Conclusion

Both DVWA and OWASP Mutillidae II were confirmed vulnerable to Local File Inclusion and SQL Injection at their default/low security settings, using only benign markers and the applications' own fictional demo data. The comparison across security levels revealed a materially important distinction: DVWA's High security level addresses these vulnerability classes only superficially (filename pattern matching for LFI; session-based delivery and result-capping for SQLi), leaving the underlying unsanitised string-concatenation root cause intact and exploitable by any attacker capable of crafting raw HTTP requests. Mutillidae's Secure level, in contrast, appears to implement genuine allow-list and parameterisation controls that fully closed both vulnerability classes without introducing new information disclosure.

The practical implication for real-world application security is twofold: first, fixed resource mappings and prepared statements are structurally superior to denylist- or pattern-based defences, because they remove the vulnerable operation from the attacker's reach entirely rather than attempting to filter a large and evolving space of malicious input. Second, automated or superficial security testing that does not manually validate boundary conditions — result-set size, response length, and error-message content — risks materially mis-rating an application's true exposure, as demonstrated by DVWA's High-security SQLi module, which can appear safe under a naive test while remaining fully exploitable.